



Security Assessment Final Report



Lido CSM v3 and CM v2

June 2026

Prepared for Lido

Table of Contents

Project Summary	3
Project Scope	3
Project Overview	3
Protocol Overview	3
Assessment Methodology	4
Threat and Security Overview	5
Assets	5
Actors	6
Roles	6
Assumptions	7
Findings Summary	8
Severity Matrix	8
Detailed Findings	9
Low Severity Issues	11
L-01: Consolidations create inconsistencies in CMv2 key balance accounting	11
L-02: Initial deposit allocator's count-based fairness math systematically biases CMv2 stake distribution.	12
L-03: Top ups to validators that are in exit queue, create 2 valid withdrawal requests	14
L-04: Missing isValidatorSlashed check in capTopUpLimitsByKeyBalance() lets stale top-up witnesses race the slashing proof path	16
L-05: keyConfirmedBalance can be stale in case of prover failure, leading to incorrect withdrawal report penalty accounting	18
Informational Severity Issues	20
I-01: ExpiredBondLockRemoved event not emitted when an expired bond lock gets overwritten by a new lock	20
I-02: Bond lockPeriod documentation overpromises auto-unlock	21
I-03: unstETH claim requires undocumented manual redemption step when rewardAddress is a passive contract	22
I-04: Extended-mode changeNodeOperatorRewardAddress() lacks ownership confirmation	23
I-05: getNodeOperatorsCount, getActiveNodeOperatorsCount and getNodeOperatorIsActive have misleading names and functionality	24
I-06: ETH bond path silently absorbs msg.value overpayment as additional bond	26
I-07: Retroactive bond curve updates can silently unbond existing Node Operators	28
Deployment Verification	29
Disclaimer	36
About Certora	36

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Lido CSM v3 and CM v2	https://github.com/lidofinance/community-staking-module	Initial: 0e2f229 Fix review: 4d3de66 Verification: 4d3de66 (CSM v3); 3b0c016 (CM v2)	EVM

Project Overview

This document describes the security review of **Lido CSM v3 and CM v2**. The work was undertaken from **March 17th, 2026** to **May 25th, 2026**.

The following contracts are included in our scope:

src except src/interfaces & src/utils

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Permissionless (CSM) and pre-vetted (CM) staking modules for Lido. Operators post bond (stETH shares), upload keys, are deposited by StakingRouter, earn fees, and bear penalties.

v3 adds 0x02 (2048 ETH) validators with a top-up flow, fee-splits, a generalized delayed penalty, bondDebt, and (CM) weight-based allocation.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

Assets

- **Bond** – the NO's collateral, stored by Accounting as stETH shares. Required to add keys, drawn down by penalties, and combined with rewards at claim.
- **Rewards** – non-distributed and non-claimed Node Operator rewards held by FeeDistributor; combined with bond before a claim.
- **Locked bond / bond debt** – amounts locked pending a delayed penalty, compensated from current bond. Any missing bond is recouped before a rewards claim.
- **In-flight exit-fee ETH** – the msg.value forwarded by Ejector (and by ValidatorStrikes for strike ejections) to TWG as the validator-exit fee, with a caller-chosen refundRecipient.
- **Validator keys & vetting state** – only vetted, depositable keys receive obtainDepositData allocations, so control over keys is control over stake allocation.
- **Node Operator control** – the manager and reward addresses; the manager performs key and operator management, the reward address receives the operator's rewards.
- **Reported validator CL balance** – top-up limits are computed from CL balance proofs, and exit penalties scale with validator balance. A stale or wrong value mis-targets penalties and caps.
- **Gate eligibility (one-time)** – each vetted address may create an NO or upgrade its type exactly once; using one bars the other. A consumed address cannot onboard again.
- **Module nonce** – the IStakingModule nonce consumed by StakingRouter, bumped on key/state changes.
- **CM allocation weights** (CM only) – MetaRegistry curve weights and group / sub-operator / external-operator membership, which drive how CuratedModule allocates stake across operators.

Actors

- **Node Operators** (and their manager / reward addresses) – untrusted.
- **Provers / arbitrary callers** – untrusted. They call the permissionless proof paths: withdrawal/slashing/balance proofs and bad-performance ejection. Bounded by the on-chain Merkle/SSZ proof checks.
- **Gates** (Permissionless / Vetted / Curated) – contract-only (in scope); the only callers that can create NOs, via `CREATE_NODE_OPERATOR_ROLE`. Vetted gates are Merkle-gated and assign the bond curve via `SET_BOND_CURVE_ROLE`.
- **StakingRouter** – contract-only (Lido core, trusted upstream); holds `STAKING_ROUTER_ROLE` and drives deposits and rewards minting.
- **Verifier** – contract-only (in scope); holds `VERIFIER_ROLE` and turns CL proofs validated against the beacon block root into module reports. Itself driven by untrusted provers.
- **CSM Oracle** (Performance Oracle, via HashConsensus quorum) – semi-trusted; computes reward distribution and strike assignment, and on quorum submits the rewards root and strikes root.
- **CSMC** (CSM Committee) – semi-trusted; an off-chain committee (multisig) overseeing CSM flows.
- **EasyTrack** – semi-trusted; applies privileged actions through motions, e.g. settling delayed penalties and submitting slashed-validator withdrawals with an off-chain-calculated penalty. That off-chain penalty value is a high-trust input.
- **Aragon Agent / Lido DAO governance** – trusted; holds `DEFAULT_ADMIN_ROLE` and can pause, penalize, manage curves, and upgrade contracts. Out of adversary scope.

Roles

Assignees per specification, grouped by trust.

- **Governance (trusted)** – `DEFAULT_ADMIN_ROLE` (Aragon Agent) grants/revokes all roles and sets recipients. Proxy upgrades are gated separately by the OssifiableProxy admin (the same Aragon Agent), not by `DEFAULT_ADMIN_ROLE`. `PAUSE_ROLE` (GateSeal one-shot plus DG Reseal Manager) and `RESUME_ROLE` (DG Reseal Manager only) can only halt or resume the protocol (an availability matter); they cannot move funds or alter accounting.

- **Contract-only** – STAKING_ROUTER_ROLE (StakingRouter) covers deposits, allocation, and rewards hooks. CREATE_NODE_OPERATOR_ROLE belongs to the three gates. VERIFIER_ROLE plus REPORT_REGULAR_WITHDRAWN_VALIDATORS_ROLE (Verifier) cover proof-gated reporting.
- **Semi-trusted** – EasyTrack holds REPORT_SLASHED_WITHDRAWN_VALIDATORS_ROLE, SETTLE_GENERAL_DELAYED_PENALTY_ROLE, and SET_TREE_ROLE. CSMC holds REPORT_GENERAL_DELAYED_PENALTY_ROLE, REWIND_TOP_UP_QUEUE_ROLE, and MANAGE_GENERAL_PENALTIES_AND_CHARGES_ROLE. SET_BOND_CURVE_ROLE is held by both CSMC and VettedGate instances. The slashed-penalty value is a high-trust input.
- **Mostly unassigned (trusted if granted)** – MANAGE_*_ROLE on ParametersRegistry sets per-curve params; note the custom modifier also lets the admin act without holding the role. Also MANAGE_BOND_CURVES_ROLE, RECOVERER_ROLE, and SET_BOND_CURVE_WEIGHT_ROLE (CM).

Assumptions

- Role holders act in good faith and roles are granted correctly. Privileged-role compromise, especially admin, is out of scope.
- The CSM Oracle quorum is honest for both the rewards root and the strikes root. Openings are verified; contents are not.
- EIP-4788 beacon roots are available and correct (the SSZ proof anchor).
- Upstream Lido core is correct: share math, Burner, StakingRouter v3 plus TopUpGateway, and TWG.
- initialize/finalizeUpgradeV3 run in the same transaction as deploy/upgrade, else a frontrun window opens.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	-	-	-
Low	5	5	1
Informational	7	7	5
Total	12	12	6

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
L-01	Consolidations create inconsistencies in CMv2 key balance accounting	Low	Acknowledged
L-02	Initial deposit allocator's count-based fairness math systematically biases CMv2 stake distribution	Low	Acknowledged
L-03	Top ups to validators that are in exit queue, create 2 valid withdrawal requests	Low	Acknowledged
L-04	Missing isValidatorSlashed check in capTopUpLimitsByKeyBalance() lets stale top-up witnesses race the slashing proof path	Low	Fixed
L-05	keyConfirmedBalance can be stale in case of prover failure, leading to incorrect withdrawal report penalty accounting	Low	Acknowledged
I-01	ExpiredBondLockRemoved event not emitted when an expired bond lock gets overwritten by a new lock	Informational	Fixed
I-02	Bond lockPeriod documentation overpromises auto-unlock	Informational	Fixed

I-03	unstETH claim requires undocumented manual redemption step when rewardAddress is a passive contract	Informational	Acknowledged
I-04	Extended-mode changeNodeOperatorRewardAddress() lacks ownership confirmation	Informational	Acknowledged
I-05	getNodeOperatorsCount, getActiveNodeOperatorsCount and getNodeOperatorIsActive have misleading names and functionality	Informational	Fixed
I-06	ETH bond path silently absorbs msg.value overpayment as additional bond	Informational	Fixed
I-07	Retroactive bond curve updates can silently unbond existing Node Operators	Informational	Fixed

Low Severity Issues

L-01: Consolidations create inconsistencies in CMv2 key balance accounting

Severity: Low	Impact: Low	Likelihood: Low
Files: src/MetaRegistry.sol	Status: Acknowledged	

Description:

During a CMv1 → CMv2 consolidation, the source key is marked exited in CMv1 (reducing the NO's external stake in `MetaRegistry::_getOperatorExternalStakeTypeNOR`) before the consolidated balance lands on the CMv2 target key. Until a balance proof is submitted, the operator's `current` validator count is understated, causing the allocator to over-allocate new deposits to that operator.

External stake for NOR-type operators is computed as:

`stake = (totalDepositedValidators - totalExitedValidators) * EXTERNAL_STAKE_PER_VALIDATOR;`

As soon as the CMv1 validator exits and is reported as exited, `totalExitedValidators` increases and `externalStake` drops. The matching balance on the CMv2 target key is only reflected after a balance proof is submitted, which depends on consolidation finality on the CL and the off-chain Prover.

During this window the allocator sees CMv1 stake already removed and CMv2 `keyConfirmedBalance` not yet increased, so the operator looks lighter than it actually is and receives a disproportionate share of new deposits.

Recommendations:

After discussion with the client, we recommend documenting this behavior, as it affects all NOs and the allocation logic will not hold true during periods of consolidation.

Customer Response:

Acknowledged. Documented as recommended.

L-02: Initial deposit allocator's count-based fairness math systematically biases CMv2 stake distribution

Severity: Low	Impact: Low	Likelihood: Low
Files: src/lib/allocator/CuratedDepositAllocator.sol	Status: Acknowledged	

Description:

`CuratedDepositAllocator._collectDepositableOperatorsData` computes per-operator footprint in validator-count units. It divides external (CMv1) stake by `MAX_EFFECTIVE_BALANCE` (2048 ETH) to convert it into an "equivalent validator count". This formula is structurally inconsistent with the post-Pectra world, where a validator's stake is no longer fixed at 32 ETH but can range from 32 ETH up to 2048 ETH, and it is inconsistent with the ETH-based math the same file uses for top-ups (`_collectTopUpGlobalBaseline`).

The count-based math breaks in two ways. First, the `externalStake / 2048` conversion under-counts CMv1 external footprints within the external-stake term itself: CMv1 validators are 0x01 (32 ETH each) but the divisor assumes 0x02 max effective balance, so an operator with 7,000 CMv1 validators (224,000 ETH external) contributes only 109 to that term instead of 7,000. The total `current` for such an operator is dominated by the under-counted external term until their internal CMv2 deposited-key count grows large enough to overshadow it. Second, the formula counts each internal CMv2 validator as a single unit regardless of its actual balance, so a fresh 32 ETH validator and a maxed-out 2048 ETH validator contribute identically to `current`.

The fairness algorithm uses `current` to compute each operator's imbalance relative to their weight-derived target. When `current` is understated, the algorithm over-allocates new deposits to operators with significant CMv1 footprints or heavily-topped-up CMv2 validators, and under-allocates to CMv2-only operators with fresh validators. The bias activates on every initial deposit allocation round and requires no adversarial behavior — it follows from the protocol running its own algorithm on the data it already has.

In practice, the per-round distortion for most operators is single-digit-percent rather than order-of-magnitude: internal CMv2 deposited-key counts grow quickly and dominate `current`

within months for active dual-module operators, shrinking the relative bias substantially. Runaway over-allocation is further bounded by `deposableValidatorsCount` (the per-operator cache cap) and the DAO-set `targetLimit` (per-operator hard cap), and the DAO can re-tune curve weights to compensate for known biases.

The Depositor Bot prioritizes predeposits over top-ups: top-ups only fire when no predeposits remain available. During ramp-up, predeposits dominate by design, and top-ups carry no corrective signal in the most important phase of CMv2's life. Even later, top-ups can equalize ETH balances but cannot reduce validator counts. An operator who received excess initial deposits will run more validators than their weight-derived target for a long time.

Recommendations:

Replace the count-based math with the ETH-based approach already used by the top-up flow's equivalent function in the same file. Mirror what `_collectTopUpEligibleOperatorsData` does:

```
-uint256 current = no.totalDepositedKeys - no.totalWithdrawnKeys;  
-if (externalStake > 0) current += externalStake / MAX_EFFECTIVE_BALANCE;  
+uint256 balance = StakeTracker.getOperatorBalance($, i);  
+uint256 currentStake = balance + externalStake;
```

Customer Response:

Acknowledged. Taking into account node operator stake in CMv1 allows a fair initial allocation during the migration phase. Even though the initial deposits might be skewed in favour of some operators due to the lower external stake estimate, the total stake allocation will be rectified by top-ups that will take into account the actual balances of node operators. Given the entry queue conditions around the module release, it sounds reasonable to seed operator keys sooner than later, given that the ultimate goal is to seed all keys provided by the node operators. We do not see a clear benefit in changing the behaviour at the current stage of development, and we tend to accept the presented side effects given the explanation above.

L-03: Top ups to validators that are in exit queue, create 2 valid withdrawal requests

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/Verifier.sol	Status: Acknowledged	

Description:

When CSM allocates a top-up to a validator whose CL exit is already in flight, the CL postpones the [deposit](#) and credits it only after [is_validator_withdrawn](#). If the sweep cursor hits the validator during epoch `withdrawable_epoch` before the epoch boundary applies the credit, the CL emits two independent withdrawal records for the same validator — the main exit and the later postponed-deposit sweep. Both pass the Verifier's checks if their amounts clear the `MIN_WITHDRAWAL_RATIO` floor and whichever passes first short-circuits any after, so first proof submitted wins. Anyone can submit either proof and the explanation below covers the scenario of using the postponed-deposit (top up) proof.

`MIN_WITHDRAWAL_RATIO` is 99% for CSM and 99.5% for the CM (a maximum tolerated shortfall of 1% / 0.5%), we'll use the CSM for this example.

1. Validator has proven CL balance 1000 ETH (so stored `keyConfirmedBalance` = 968, `minExpectedBalance` = 1000).
2. Exit is initiated, either due to strikes or voluntary. Validator is placed in the exit queue on the CL.
3. SR calls `allocateDeposits` and this key gets the remainder of `maxDepositAmount`, in the example this is 992 ETH. `capTopUpLimitsByKeyBalance` only filters `isValidatorWithdrawn`, not exit-initiated. (this can also happen with an SR given `topUpLimit` = 992 ETH)
4. On the CL `process_pending_deposits` sees `is_validator_exited == true` → deposit added to `deposits_to_postpone`.
5. During `withdrawable_epoch` the sweep cursor lands on the validator when processing withdrawals and processes his original 1000 ETH, his balance goes to 0 here.

6. At the start of `withdrawable_epoch + 1`: `process_pending_deposits` credits the postponed 992 ETH, so balance jumps from 0 to 992 ETH.
7. The next time the sweep cursor hits the same validator he sees he is withdrawable again and will emit a second withdraw, balance goes back to 0.
8. Adversary submits `processWithdrawalProof` with the proof of the step-7 record (the top up withdraw):
 - `PartialWithdrawal` check $992 < (1000 \times 0.99) = 992 < 990$. False, so the check passes
9. `_fulfillExitObligations`: `exitBalance` = 992 < `minExpectedBalance` = 1000, 8 ETH penalty charged to NO bond.

If the original 1000 ETH proof was used, then no penalty would have been applied at all.

The issue is similar to [LIP-29](#) where the attacker has to fund the malicious deposit, here the trigger is funded by the protocol itself.

Recommendations:

There is no clean fix due to the design of the system. Our recommendation is to document this similar to LIP-29.

Customer Response:

Acknowledged. The issue is documented in [LIP-33](#).

L-04: Missing `isValidatorSlashed` check in `capTopUpLimitsByKeyBalance()` lets stale top-up witnesses race the slashing proof path

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/lib/NodeOperatorOps.sol	Status: Fixed	

Description:

CSM's `capTopUpLimitsByKeyBalance` zeroes the per-key top-up limit for withdrawn validators but not for slashed ones. When a slashed validator sits at the top-up queue head and a top-up transaction lands with a stale-but-valid beacon witness, the on-chain code allocates Lido buffer ETH to a validator the CSM contract already knows is slashed.

`TopUpGateway._evaluateTopUpLimit()` returns 0 for slashed validators based on a Merkle-verified `ValidatorWitness`. That witness reflects the validator's state at the slot of the beacon root used to build it, not at execution time. `_verifyRootAge()` accepts any root within `maxRootAge` (300 seconds in the testnet TOML and unit-test default). Inside that window, a witness built before a slashing event remains cryptographically valid and reports `slashed = false` even after the validator has actually been slashed on the beacon chain.

`Verifier.processSlashedProof()` is permissionless. The moment any caller submits a slashing proof against a post-slash beacon root, CSM's `$.isValidatorSlashed[pointer]` flips to `true`. From that point onward, CSM holds fresher information than any pre-slash witness still in flight. But `capTopUpLimitsByKeyBalance` does not consult that flag, so a top-up carrying a stale witness passes the cap and the queue head dequeues with a real ETH allocation to the slashed validator.

Impact is bounded by Ethereum's Pectra spec. In `process_pending_deposits`, deposits targeting a validator with `exit_epoch != FAR_FUTURE_EPOCH` are postponed back into `state.pending_deposits` every epoch until `withdrawable_epoch`. So the ETH is not exposed to inactivity, correlation, or other balance-scaled penalties – it sits idle for up to ~36 days and is then applied at `withdrawable_epoch` and immediately swept to the withdrawal address. There is no principal loss but Lido's buffer takes a liquidity hit and CSM's `keyAllocatedBalance` records the top-up as productive when it is in fact deferred on the consensus layer.

Likelihood is low. The race window is `maxRootAge` (5 minutes as currently configured), it only fires when a slashing event lands inside that window and a slashing proof is submitted before the bot's batch executes, and Lido's in-house prover uses finalized state and so will not race itself. The exploitable case requires a third-party fresh-proof submission, which the Lido team confirmed is unlikely in steady-state operation.

Recommendations:

Mirror the existing `isValidatorWithdrawn` short-circuit:

```
function capTopUpLimitsByKeyBalance(...) external returns (uint256[] memory
cappedTopUpLimits) {
    // ...
    for (uint256 i; i < len; ++i) {
        uint256 pointer = KeyPointerLib.keyPointer(operatorIds[i],
keyIndices[i]);
        if ($.isValidatorWithdrawn[pointer]) continue;
        if ($.isValidatorSlashed[pointer]) continue;    // <- check to be added
        // ...
    }
}
```

Customer Response:

Fixed as of [403c2bd](#)

Fix Review:

Fix confirmed

L-05: `keyConfirmedBalance` can be stale in case of prover failure, leading to incorrect withdrawal report penalty accounting

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/Verifier.sol	Status: Acknowledged	

Description:

When `slashingPenalty` is not explicitly set, the module falls back to:

```
else if (validatorInfo.exitBalance < minExpectedBalance) {
    penaltySum += minExpectedBalance - validatorInfo.exitBalance;
}
```

where `minExpectedBalance` is `ValidatorBalanceLimits.MIN_ACTIVATION_BALANCE + keyConfirmedBalance`

The `penaltySum` is then penalized from the NO's bond, thus a greater `minExpectedBalance` compared to `exitBalance` (which is the validator's balance on the CL) means a larger penalty.

`keyConfirmedBalance` is a monotonically increasing value and it's only increased through `processBalanceProof` and `processHistoricalBalanceProof`. The two functions are permissionless, but are meant to be called by Lido's Prover bot and NOs are incentivized to run their own "prover bots", in order to update their validator keys balances. Balance updates are meant for top-ups and consolidations, as there is no way to prove them on-chain only. The issue is that NOs are actually incentivized to *not* update their balances, a greater difference between `keyConfirmedBalance` and `exitBalance` means a larger penalty. The example below illustrates the slashed-with-zero-`slashingPenalty` case, but the same mechanism applies to any non-slashed exit with a real loss (e.g., voluntary exit after inactivity leak), since `isSlashed=false` flows through the same `else if` branch.

1. Validator A has 1000 ETH on the CL, but his `keyConfirmedBalance = 800 ETH`
2. Validator A gets slashed and withdraws from the CL, his CL balance is now 900 ETH.
3. Validator A is reported as withdrawn, but his `penaltySum = 0` because `exitBalance > keyConfirmedBalance`, thus no penalty is applied. Reliance on `keyConfirmedBalance`

introduces a single point of failure, the value is only reported from the Prover bot, we established that NOs are disincentivized to report their balances, if it goes offline or doesn't report in a timely manner, all the losses are socialized from Lido users.

Mitigated in practice by the csm-prover-tool's pre-withdrawal balance update in `handleWithdrawalsInBlock`, which submits `processBalanceProof` before each `processWithdrawalProof` to ensure `keyConfirmedBalance` is current at penalty calculation time. Severity Low, contingent on this off-chain mitigation remaining in place; would be Medium without it.

Recommendations:

No elegant fix on chain exists due to the system design. Our recommendation is to document prover failures, the consequences and hardening of the off-chain component

Customer Response:

Acknowledged. It's hard to argue that the modules rely so heavily on proofs delivered via the Verifier contract. That's why there will be several prover instances run by the protocol to ensure bot liveness. It is also worth mentioning the specific [rule](#) in the prover bot that triggers once a validator is withdrawn to deliver its balance.

Informational Severity Issues

I-01: ExpiredBondLockRemoved event not emitted when an expired bond lock gets overwritten by a new lock

Files:

src/abstract/BondLock.sol

Description:

When `_lock()` is invoked on an NO whose previous lock has expired but `unlockExpiredLock` has not yet been called, the expired amount is silently dropped: the `if (currentLockUntil > block.timestamp) amount += lock.amount` guard excludes the expired portion from the new sum, and `_changeBondLock()` overwrites the storage slot with the new lock data. Only `BondLockChanged` event is emitted; no `BondLockRemoved` or `ExpiredBondLockRemoved` event fires for the discarded amount.

On-chain state is correct. However, an off-chain consumer cannot reconstruct lock history from events alone.

Recommendations:

Emit `ExpiredBondLockRemoved(nold)` from `_lock()` before overwriting an expired lock, so event-replay consumers can identify implicit cleanup.

Customer Response:

Fixed as of [403c2bd](#).

Fix Review:

Fix confirmed

I-02: Bond lockPeriod documentation overpromises auto-unlock

Files:

src/abstract/BondLock.sol

Description:

`BondLock._lock()` sums a new lock amount with any active existing lock and sets `until = max(now + bondLockPeriod, currentUntil)`. A new report of any size effectively refreshes the existing lock's deadline by a fresh `bondLockPeriod`.

The project docs state: *"If the penalty is not settled for the `lockPeriod`, the locked bond is automatically unlocked."* `settle()` is a defined operation and a new report is not a settle, so the doc's literal wording implies the old portion should auto-unlock regardless of subsequent lock reports. The `BondLock` NatSpec uses the same single-lock framing.

The team confirmed that auto-expiry is a dead man's switch safety fallback, not a fixed NO countdown so the current implementation matches intent but documentation does not.

Recommendations:

Rewrite `penalties.md` to describe `bondLockPeriod` as a dead man's switch. Note that active reporting prolongs locks indefinitely.

Additionally, update `BondLock.sol` NatSpec to reflect the stacking semantics.

Customer Response:

Fixed as of [403c2bd](#).

Fix Review:

Fix confirmed

I-03: unstETH claim requires undocumented manual redemption step when rewardAddress is a passive contract

Files:

src/Accounting.sol#L247

src/abstract/BondCore.sol#L121

Description:

`claimRewardsUnstETH` mints an unstETH ERC-721 NFT to the node operator's `rewardAddress` via `WithdrawalQueue.requestWithdrawals`. The NFT must later be redeemed for ETH in a separate transaction, by its owner calling `WithdrawalQueue.claimWithdrawal(requestId)`. The unstETH claim is inherently two-step: request, then redeem.

Lido's official DVT setup guides ([Obol](#), [SSV](#)) instruct operators to set `rewardAddress` to a `OxSplits` Splitter contract for pooled cluster payouts. The Splitter accepts the NFT (it inherits `ERC721Holder`) but has no autonomous path to call `claimWithdrawal` – its `distribute*` functions only operate on ETH and ERC-20 balances. Redemption requires the Splitter's `owner` to call `SplitWalletV2.execCalls(Call[])` with a payload invoking `claimWithdrawal`, sending the ETH back to the Splitter for downstream distribution.

The CSM widget already warns operators that selecting "ETH (Withdrawal NFT)" with a contract `rewardAddress` is not recommended, which covers the primary interaction path. The remaining gap is at the setup-side documentation where the `OxSplits` configuration is introduced, and for direct integrators that do not pass through the widget UI.

Recommendations:

Extend the existing widget warning into the Obol and SSV DVT setup guides where `rewardAddress = OxSplits` is recommended. A one-liner noting that `claimRewardsUnstETH` requires the Splitter owner to invoke `execCalls(Call[])` with a `WithdrawalQueue.claimWithdrawal` payload after finalization would round out the coverage for setup-side readers and direct integrators.

Customer Response:

Acknowledged. We will add the explainer in the documentation or on the widget.

I-04: Extended-mode `changeNodeOperatorRewardAddress()` lacks ownership confirmation

Files:

src/lib/NOAddresses.sol

Description:

`changeNodeOperatorRewardAddress` is the extended-mode shortcut that lets an NO's manager unilaterally point the reward address at a new destination. Unlike every other address change in `NOAddresses.sol`, it is a one-step process.

The asymmetry tracks the trust model: in extended mode the reward address holds no authority, so the "new authority-holder must opt in by demonstrating they hold the keys" purpose of the two-step pattern does not apply.

However, typo defense still applies. A mistake by the manager commits a bad reward address immediately and any rewards claimed before the operator notices land at an unrecoverable destination. Damage is bounded as the manager can call `changeNodeOperatorRewardAddress` again to fix the destination. But each typo creates a financial-loss window proportional to claim cadence.

Recommendations:

Consider adding an opt-in two-step variant for extended-mode reward changes (`extendedProposeRewardAddressChange` + `confirmRewardAddressChange`). Existing one-step path remains for operators who want it.

Customer Response:

Acknowledged. This method exists to cover, among other things, the case when the rewards address is a contract that cannot be used to confirm the proposal, e.g. splitter contracts and similar.

I-05: `getNodeOperatorsCount`, `getActiveNodeOperatorsCount` and `getNodeOperatorIsActive` have misleading names and functionality

Files:

`src/abstract/BaseModule.sol`

Description:

`BaseModule` exposes these three view functions:

```
function getNodeOperatorsCount() external view returns (uint256) {
    return _baseStorage().nodeOperatorsCount;
}

function getActiveNodeOperatorsCount() external view returns (uint256) {
    return _baseStorage().nodeOperatorsCount;
}

function getNodeOperatorIsActive(uint256 nodeOperatorId) external view returns
(bool) {
    return nodeOperatorId < _baseStorage().nodeOperatorsCount;
}
```

The system has no concept of active/inactive NOs — and `getActiveNodeOperatorsCount` returns the exact same value as `getNodeOperatorsCount`. `getNodeOperatorIsActive` is also the same as the existence check in `_onlyExistingNodeOperator`, which itself reverts with `NodeOperatorDoesNotExist`.

The names communicate semantics that the contract does not implement, which can mislead integrators reading the interface.

Recommendations:

Rename the functions to correctly communicate their functionality and remove either `getNodeOperatorsCount` or `getActiveNodeOperatorsCount` as they return the same values.

Customer Response:

Fixed as of [403c2bd](#).

Fix Review:

Fix confirmed

I-06: ETH bond path silently absorbs msg.value overpayment as additional bond

Files:

src/abstract/BaseModule.sol

Description:

`addValidatorKeysETH` requires `msg.value >= _getRequiredBondForNextKeys(...)` and forwards the entire `msg.value` to `Accounting.depositETH`, which submits all of it to Lido and credits the resulting stETH shares to the operator's bond:

```
if (msg.value < _getRequiredBondForNextKeys(accounting, nodeOperatorId,
keysCount)) revert InvalidAmount();
if (msg.value != 0) accounting.depositETH{ value: msg.value }(from,
nodeOperatorId);
```

Any surplus over the required bond is silently absorbed as extra bond rather than refunded. The stETH/wstETH paths pull exactly `required` via `transferSharesFrom` / `transferFrom`, so there is an asymmetry between the ETH and stETH/wstETH paths: ETH accepts `>= required` and stETH/wstETH pull exactly `== required`.

No funds are lost and the surplus is recoverable through the standard bond-claim flows. But the behavior is non-obvious and has two practical consequences: (1) the surplus is converted to stETH at the current share rate and exposed to stETH/ETH price drift until claimed, and (2) integrators that pass a safety buffer on the ETH path (a reasonable defense against share-rate rounding) silently over-bond on every call.

Recommendations:

Document the behavior in NatSpec on `addValidatorKeysETH` / `addNodeOperatorETH`.

Refunding the excess on-chain adds gas and a re-entrancy surface that does not exist today so we would advise against that approach. Updating the documentation is the cheaper fix.

Customer Response:

Fixed as of [403c2bd](#).

Fix Review:

Fix confirmed

I-07: Retroactive bond curve updates can silently unbond existing Node Operators

Files:

src/Accounting.sol

Description:

`Accounting.updateBondCurve(curveId, newIntervals)` modifies curve data in-place for a given curve ID. All NOs currently assigned to that curve see their bond requirements change immediately. A previously fully-bonded NO can become unbonded and face forced exit via VEBO without any misbehavior on their part.

The `MANAGE_BOND_CURVES_ROLE` is not assigned by default and requires explicit governance action to grant. However, the role is expected to be assigned in the future and the spec does not document the retroactive impact on existing NOs.

Recommendations:

Document in the spec that `updateBondCurve` affects all existing NOs on the curve retroactively.

The recommended operational procedure when tightening bond requirements should be:

1. Create a new curve with stricter terms via `addBondCurve`.
2. Notify affected NOs and provide a top-up window.
3. Migrate NOs individually via `setBondCurve` once they have had time to adjust.

This avoids in-place mutation of existing curves and gives NOs a grace period before enforcement.

Customer Response:

Fixed as of [403c2bd](#).

Fix Review:

Fix confirmed

Deployment Verification

Following the security assessment, Certora performed an independent verification of the mainnet deployment of the reviewed contracts. The objective was to confirm that the bytecode deployed to Ethereum mainnet corresponds to the audited source, that all externally-linked libraries are deployed and correctly linked, and that the deploy-time configuration and privilege handoff match the reviewed specification.

Two deployments were in scope: the Curated Module v2 (CM v2), deployed from scratch, and the CSM v3 implementations, deployed for the in-place upgrade of the existing CSM instance.

Verification was performed against live mainnet state using the Foundry toolchain ([cast](#)), a fresh compilation of the audited source, an Ethereum RPC endpoint, and Etherscan. This section documents the deploy-phase state only; activation of both deployments is performed by a subsequent governance vote and is out of scope (see Activation).

Scope and Method

For each deployment, the following checks were performed:

- **Bytecode equivalence** – The deployment commit was recompiled and the resulting deployed/runtime bytecode of every implementation and library was compared against the on-chain code ([cast code](#)), after substituting the deployed library addresses into the linked placeholders and masking immutable byte-ranges. Immutable values were then checked separately via their public getters.
- **Library deployment and linking** – Every externally-linked library was confirmed to be deployed as a standalone contract, and each consumer was confirmed to embed the correct library address at every link site, with no unresolved link placeholders.
- **Proxy, roles and configuration** – Proxy configuration, the AccessControl privilege handoff, and the load-bearing deploy-time configuration (bond parameters, Merkle roots, oracle parameters) were verified via on-chain reads.

Provenance and Bytecode Equivalence

The contracts deployed to mainnet are built from source identical to the audited codebase:

- **CSM v3 implementations** – [4d3de66](#) (The audited commit)
- **Curated Module v2** – [3b0c016](#) (Descendant of the audited commit; the src/**/*.sol diff is empty – the contract source is identical, and the delta consists only of gate participant lists)

Result – all deployed bytecode matches the audited source:

- **CSM v3** – 13 contracts ([CSModule](#), [Accounting](#), [FeeOracle](#), [FeeDistributor](#), [ParametersRegistry](#), [ValidatorStrikes](#), [ExitPenalties](#), and [VettedGate](#) implementations, plus the new [Verifier](#), [Ejector](#), [PermissionlessGate](#), [OneShotCurveSetup](#), and [MerkleGateFactory](#)) and 9 external libraries match byte-for-byte. The [IdentifiedDVTClusterGate](#) proxy resolves to the verified [VettedGate](#) implementation.
- **CM v2** – 11 implementations and 8 external libraries match byte-for-byte.
- **Library linking** – every consumer embeds the correct deployed library address at each link site; no unresolved placeholders remain; shared libraries (for example [AssetRecovererLib](#), reused across both deployments) resolve consistently everywhere.

Configuration and State

- **Merkle roots** – All gate tree roots are final values. Gates intended to onboard operators at launch hold real roots; gates not yet active hold the documented "disabled" sentinel ([0x111...1](#)). No placeholder roots remain.
- **CircuitBreaker** – The configured [CircuitBreaker](#) ([0x6019CB55...](#), per LIP-34) is a deployed contract, and both deployments reference it.
- **Privilege handoff** – Every newly-deployed contract has had [DEFAULT_ADMIN_ROLE](#) transferred to the Aragon Agent and the deployer's rights revoked; the proxy admin is the Aragon Agent. The deployer retains no privileges over the deployed contracts.
- **Wiring and parameters** – Inter-contract immutables (for example module [Accounting](#), [Verifier](#) [->](#) module) resolve to the deployed addresses; [MIN_WITHDRAWAL_RATIO](#) ([9900](#) for CSM, [9950](#) for CM), the default keys limit, and the oracle consensus version match the audited configuration.
- **Upgrade not yet applied** – Consistent with the deploy-phase state, the existing CSM proxy still points to its previous implementation; the CSM v3 implementations are deployed but not yet wired into the live proxies.

Validated Contract Addresses

The following addresses were confirmed on Ethereum mainnet to hold bytecode identical to a fresh build of the audited source, using the method described above. External dependencies referenced by the deployment but outside the audited codebase (the CircuitBreaker and the LidoLocator) are excluded.

Curated Module v2

Core contracts – both proxy and implementation validated:

Contract	Deployment
CuratedModule	Proxy: 0xDA5F930cE326EB5205085D66c72A4E79d60cB8C1 Implementation: 0x959fC67FE53c8A6C7a1AE73430Aa07a36eD9337
Accounting	Proxy: 0x2F91e3A8C5d6593bf4F8403fCfeCcd62dF59f6F6 Implementation: 0xB41F5d2721906b3BE4fC7ae08261266C801076C2
FeeOracle	Proxy: 0x8EeFCdbD984c30E472BcbF545783D051CB5114e5 Implementation: 0x16804084408B6Caad10046F49aD421cBA56C0b5e
FeeDistributor	Proxy: 0x367d23c756599c20DCc8D6943F4976E8F88D60d7 Implementation: 0x7C8FEE1dcC95Df60fC9b5BE7603c28Eb3af16753
Parameters Registry	Proxy: 0xffC1C5d59CeAC6F6c27E701F04a70cb50474607C Implementation: 0xFF419BbBC5f44d46547079922a88d691886d192a
ValidatorStrikes	Proxy: 0xf4618370a1fBf46905B16C10817c8CFaD924D6db Implementation: 0x239Ee6ab18fB06370ad53Ce05097B32208fB0a30
ExitPenalties	Proxy: 0x004aFb7DAA7dEA20EbAaB75c9F4892C879FaCCe0 Implementation: 0x3766ABbA4635EE0fC3E6A7EB3EF169fe930df9c2
MetaRegistry	Proxy: 0xA64b339eebD3dC3De848298B6a140955932901d8 Implementation: 0x6d852907463496622bb5FE5bc55cc30C4682E10e

Non-upgradeable contracts:

Contract	Address
Verifier	0xC392F457960f1B13Ebaf1aa6C065479dD507E1E3
Ejector	0xe181A377A2d2BDE9A83f1474BC3DB7A412de091E
CuratedGate (implementation)	0x3cb948FD454ad6b20DE67633f25DcbDbEaa0e849
CuratedGateFactory	0xDdE99d63b352A665d04339D4792E6852Ce89d1B7
HashConsensus	0x902D64c93F6595339aA46105627a085591051aFb

Curated gates (proxies on the CuratedGate implementation above):

Contract	Address
Professional Operator Gate	0x6093EFA6B5E2FF3be54d1c895c9deA932805c49F
Professional Trusted Operator Gate	0x8c002c6eE10cf8adb78D1F9EB2e134FdaF8A7C1a
Public Good Operator Gate	0x207798e6fD1aa7Ee8a63782A64c959cD6727b78C
Decentralization Operator Gate	0xeF273Ca4A21Ba7B414Ae3C9f9b443038cb133F72
Extra Effort Operator Gate	0x3BbBb175f7F07954DE00052b20E1c5572223F24D
Intra-Operator DVT Cluster Gate	0x86A8d4E0db5938D21d98047544668FCCB1A9ADc8
Intra-Operator DVT Cluster Plus Gate	0x773933F9db8964A17d62fb808f2EC7A2de4247CC

External libraries:

Contract	Address
AssetRecovererLib	0x37aDa408AE3c3992953688e2CCb9eE7a3dfdA902
BondCurvesLib	0xC4511d09639e5E174506083443da230D39196323
NodeOperatorOps	0xDD42EE5D54A1822021782F3F455bb99fBC19499A
NOAddresses	0x9D9c8799189c797f6e2dA74F71aDF84492adA7D3
StakeTracker	0xbb6E4Db18182d45038F91B9F1195291c206fd8d2
WithdrawnValidatorLib	0x3bf9674f062aF9BA94FdAe9Fcdf2D0001FFf0a3A
GeneralPenaltyLib	0xF05545ED71c60bBba6E73B6B70B15D4f5F22C0f4
CuratedDepositAllocator	0xa4fCD4dDa0e4a847142E3592C97c77d8B9B3Cf5F

CSM v3

Implementations deployed for the in-place upgrade. The proxies are pre-existing and would be repointed to these implementations by the governance vote; the implementation addresses below are the ones validated against the audited source. The existing proxies were confirmed to still point to their previous implementations (i.e. the upgrade has not yet been applied).

Contract	Deployment
CSModule	Proxy: 0xdA7dE2ECdDfccC6c3AF10108Db212ACBBf9EA83F Implementation: 0x63992a86f009fcC796a8369feEfB68880aef4e3a
Accounting	Proxy: 0x4d72BFF1BeaC69925F8Bd12526a39BAAb069e5Da Implementation: 0xe768572cc5aE5C698345C59288d871a949Ea8bd3
FeeOracle	Proxy: 0x4D4074628678Bd302921c20573EEa1ed38DdF7FB Implementation: 0xecE6e0Cde61078F76b66Ef0C338a6875E5D01F79
FeeDistributor	Proxy: 0xD99CC66fEC647E68294C6477B40fC7E0F6F618D0 Implementation: 0x936da7cDB7eed1084d294E23eA1d7Ad72DCcfE0E

Parameters Registry	Proxy: 0x9D28ad303C90DF524BA960d7a2DAC56DcC31e428 Implementation: 0x107d287F178cD54792614d7D63C47D8242240BeD
ValidatorStrikes	Proxy: 0xaa328816027F2D32B9F56d190BC9Fa4A5C07637f Implementation: 0xd25E7C3923d2e68c325980b0e15eD20d62B2691F
ExitPenalties	Proxy: 0x06cd61045f958A209a0f8D746e103eCc625f4193 Implementation: 0xA5b9e96E951089E629Ab0834AEaF242a81394EA0
VettedGate	Proxy: 0xB314D4A76C457c93150d308787939063F4Cc67E0 Implementation: 0x66ADb8b3F58d3DFdF6bAdB595E41f19e947E5c14

Newly deployed contracts:

Contract	Address
Verifier (v3)	0xfce7aB839e55de77730716D05b3553e45ab3A5Ba
Ejector	0x610B517D380f287c239C93F8eF6FfBd567AA4bA5
PermissionlessGate	0xb8cd8F059Ad7a5dB8CAfDe34aAb007317F7156C8
Identified DVT Cluster Gate (proxy)	0xa12760721A72A7199aB38059DA6690b9Cd4ed7B8
Identified DVT Cluster curve setup	0x711985E069f4d702e0457C0dACAd3D3894Ce4E3
VettedGateFactory	0xc0f110Af6eA9119037a71C84D14506A22AE43DdE

External libraries (CSM-specific):

Contract	Address
DepositQueueOps	0xb430AA6C70A352c2aaC9813AE049A210dB11aB41
TopUpQueueOps	0xdA104f5f2a18405fC7cCD6E0A7FEB5B824843606

The remaining seven libraries linked by the CSM v3 contracts (AssetRecovererLib, BondCurvesLib, NodeOperatorOps, NOAddresses, StakeTracker,

WithdrawnValidatorLib, GeneralPenaltyLib) are the same deployed instances listed under Curated Module v2 above, reused across both deployments.

Activation (Out of Scope)

Both deployments are in a pre-activation state: deployed, configured, and handed to governance, but not yet active in the protocol. Activation (the proxy upgrades and `finalizeUpgradeV3` calls, the role migration from the superseded contracts, the addition of CM v2 to the `StakingRouter`, module resume, and `CircuitBreaker` pauser registration) is executed by a Lido governance vote and is outside the scope of this deployment verification.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.